CHAPTER 8

Routing: Distance Vector & Link State

Bellman-Ford tables · count-to-infinity · split horizon · link-state flooding · Dijkstra · RIP, OSPF, BGP

QUICK TRICK

DV: each router tells *neighbours* its *whole table*. Update: $\mathbf{D_x(y) = \min_v [\, c(x,v) + D_v(y) \,]}$. **LS:** each router tells *everyone* the cost of *its own links*, then runs **Dijkstra**.

8.1 Routing vs forwarding

● LEVEL 1 · BEGINNER

Forwarding = moving one packet to the next hop using the table (fast, per packet). **Routing** = building the table (slow, by an algorithm). Routes can be static (typed by hand) or dynamic (learned by a protocol). Dynamic protocols work inside an AS (RIP, OSPF) or between ASes (BGP).

Analogy. Forwarding is following road signs at a junction. Routing is the map-making that decided where the signs point.

8.2 Distance vector (DV)

● LEVEL 2 · EXAM

- Each router keeps a table: destination → (cost, next hop). It sends this vector to its **direct neighbours** periodically and on change.
- On receiving a neighbour v's vector, router x updates: $\mathbf{cost(x \to y) = \min_v [\, c(x,v) + D_v(y) \,]}$ (Bellman-Ford).
- Knows only *distances* and *next hops*, not the full map. "Routing by rumour".

Example update. X has neighbours Y (cost 2) and Z (cost 5). Y says "D is 4 away", Z says "D is 1 away". X computes min(2 + 4, 5 + 1) = **6** (tie; either next hop).

Row of a table. Use the network below (edges: A-B 2, A-C 5, B-C 1, B-D 4, C-D 2, C-E 6, D-E 1). Router C gets converged vectors from neighbours. To reach **E**: via A 5 + 6 = 11, via B 1 + 4 = 5, via D 2 + 1 = **3**, via E 6 + 0 = 6. So C → E costs 3 via D.

Count-to-infinity

Line network A — B — C with costs 1 and 1. Both A and B know how to reach C. **The link B–C fails.** B hears from A "C is 2 away" (A's old route, which actually goes through B) and wrongly decides C is 3 away via A.

Round	B says C =	A says C =	What happens
0	1	2	normal
1	3 (via A)	2	link B–C fails, B believes A
2	3	4 (via B)	A believes B
3	5	4	B believes A
4	5	6	A believes B
…	… counting up …		stops at "infinity" (16 in RIP)

Fix	How it works
Split horizon	Do not advertise a route back out of the interface you learned it from.
Poison reverse	Advertise that route back with cost = infinity.
Triggered updates	Send an update immediately when a route changes.
Hold-down timers	Ignore worse news about a route for a while.
Small infinity	RIP treats 16 as unreachable (so max 15 hops).

COMMON MISTAKE

- Split horizon / poison reverse stop two-node loops only; larger loops can still count to infinity.

8.3 Link state (LS)

● LEVEL 2 · EXAM

- Each router measures the cost to its neighbours, builds a **link-state advertisement (LSA)** and **floods** it to every router (with a sequence number so duplicates die).
- Now every router has the **same complete map**. Each runs **Dijkstra** from itself and builds its forwarding table.
- Dijkstra: keep a set of settled nodes; repeatedly settle the unsettled node with the smallest tentative distance, and relax its neighbours.

Dijkstra from A on the network above (tentative cost, shown with the node that just got settled):

Step	Settled	B	C	D	E
0	A (0)	2	5	∞	∞
1	B (2)	–	3 via B	6	∞
2	C (3)	–	–	5 via C	9
3	D (5)	–	–	–	6 via D
4	E (6)	–	–	–	–

Result at A: B = 2, C = 3, D = 5, E = 6, and **every next hop is B** (path to E: A-B-C-D-E).

	Distance vector	Link state
Tells whom	Neighbours only	All routers (flooding)
Tells what	Whole table (distances)	Own links (costs)
Knowledge	Distances and next hops	Full topology
Algorithm	Bellman-Ford	Dijkstra
Convergence	Slow, count-to-infinity	Fast, no count-to-infinity
Cost	Light on CPU and memory	More memory and CPU, more traffic at start
Examples	RIP	OSPF

8.4 Protocols in one table

● LEVEL 1 · BEGINNER

Protocol	Type	Metric	Runs over	Note
RIP	DV, intra-AS	Hop count (max 15)	UDP 520	Updates every 30 s
OSPF	LS, intra-AS	Cost (bandwidth based)	IP directly (protocol 89)	Areas, Dijkstra
BGP	Path vector, inter-AS	Policy, AS path	TCP 179	Glue of the Internet

8.5 Common mistakes

COMMON MISTAKE

- Forgetting to add the link cost to the neighbour **to** the neighbour's advertised distance.
- Saying DV floods to all routers (it speaks only to neighbours).
- Thinking Dijkstra needs the whole map in DV (it needs it in **LS**).
- Stopping Dijkstra before all nodes are settled, or settling a node that is not the current minimum.

QUICK TRICK

Memory line: "DV: tell neighbours everything I know. LS: tell everyone what I see."

GATE QUICK CHECK — Chapter 8

```
DV : D_x(y) = min_v [ c(x,v) + D_v(y) ]     Bellman-Ford     RIP max 15 hops
LS : flood LSAs -> full map -> Dijkstra      OSPF    BGP = path vector, TCP 179
Fixes for count-to-infinity: split horizon, poison reverse, triggered update, hold-down
```

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. Which algorithm does DV use and which does LS use?

Q2. What is the largest usable hop count in RIP?

Q3. Which protocol is path vector, and over which transport?

Q4. Which protocol floods advertisements to every router?

● **LEVEL 2 · EXAM**

Q5. X has neighbours Y (cost 2) and Z (cost 5). Y advertises 4 to D and Z advertises 1 to D. X's cost to D?

Q6. Using the network in 8.2, what is C's cost and next hop to reach A?

Q7. Router R receives from neighbour N (R–N cost 3): {A: 4, B: ∞, C: 2}. R currently has A: 10 via X, B: 6 via Y, C: 5 via X. Updated table?

Q8. Shortest path cost from A to E in the 8.2 network.

● **LEVEL 3 · GATE**

Q9. In the A–B–C example, B–C fails. After the first update what does B think the cost to C is?

Q10. How does split horizon prevent the loop above?

Q11. Why does link state avoid count-to-infinity?

Q12. Time complexity of Dijkstra with an array and with a binary heap (V nodes, E edges)?

Answers

Q	Solution
1	DV: **Bellman-Ford**. LS: **Dijkstra**.
2	**15** (16 means unreachable).
3	**BGP**, over **TCP** port 179.
4	**Link state** (OSPF).
5	min(2 + 4, 5 + 1) = **6**.
6	via A: 5; via B: 1 + 2 = **3**; via D: 2 + 5 = 7; via E: 6 + 6 = 12. Cost **3 via B**.
7	A: 3 + 4 = 7 < 10, so **A = 7 via N**. B: ∞ is no help, stays **6 via Y**. C: 3 + 2 = 5, equal, so **stays 5 via X**.
8	**6** via A-B-C-D-E.
9	**3** (1 + A's claim of 2).
10	A learned its route to C from B, so A **does not advertise C back to B**. B cannot be tricked into using A.
11	Every router has the full, consistent map and computes shortest paths itself, rather than trusting neighbours' distances.
12	**$O(V^2)$** with an array; **O(E log V)** with a heap.

Next: Chapter 9 — IPv4 Addressing, CIDR and Subnetting.